

Q/KXY

云计算开源产业联盟技术文件

Q/KXY CT002—2025

智能化软件测试能力成熟度模型

Maturity Model for Intelligent Software Testing Capability

2025-01-06 发布

2025-02-17 实施

云计算开源产业联盟

目 次

目 次	I
前 言	II
智能化软件测试能力成熟度模型	1
1. 范围	1
2. 规范性引用文件	1
3. 术语和定义	1
3.1	1
3.2	1
3.3	1
4. 缩略语	2
5. 智能化软件测试能力成熟度模型总体架构	2
6. 能力分级	2
6.1. 分级概述	2
6.2. 分级规则	3
7. 服务能力	3
7.1. 需求分析	3
7.2. 测试计划	5
7.3. 测试设计	6
7.4. 测试执行	10
7.5. 报告与缺陷	11
7.6. 回归测试	13
7.7. 测试准入准出评审	14
8. 模型服务	15
8.1. 模型适配	15
8.2. 模型调度	16
8.3. 模型微调	17
8.4. 安全管控	18
9. 测试监控	18
9.1. 测试执行监控	18
9.2. 模型监控	19
9.3. 资源监控	20
10. 资源管理	20
10.1. 工具管理	20
10.2. 环境管理	22
10.3. 数据管理	22
11. 知识库建设	23

前 言

本文件按照GB/T 1.1—2020《标准化工作导则 第1部分：标准化文件的结构和起草规则》的规定起草。

本文件由中国通信标准化协会提出并归口。

本文件起草单位：中国信息通信研究院、中国农业银行股份有限公司、中国邮政储蓄银行股份有限公司、中国光大银行股份有限公司、人保信息科技有限公司、东软集团股份有限公司、东方证券股份有限公司、北银金融科技有限责任公司、中移物联网有限公司、中移（杭州）信息技术有限公司、中移（苏州）信息技术有限公司、湖北华中电力科技开发有限责任公司、苏州洞察云信息技术有限公司、联想（北京）有限公司、中移动信息技术有限公司、联通数字科技有限公司、咪咕文化科技有限公司、福建福诺移动通信技术有限公司。

本文件主要起草人：刘之莹、郑立、陈屹力、栗蔚、王晓昕、黄莺、张世哲、殷坤、蒋其恩、杨忠琪、赵丽艳、邹鹏、于灏、蔡景耀、朱丽娜、张莹莹、高煜红、张志红、赵新想、王健、胡宗棠、王梦迪、顾钧、郑炎、程果、余瑾华、郭一平、郭珊宜、梁庭、胡苏、顾明、李伯龙、张强、邵荣伟、邱新启、杨建东、姚瑞祥、魏小明、严健、张力、魏少文等人。

智能化软件测试能力成熟度模型

1. 范围

本标准规定了智能化软件测试能力成熟度模型分级要求，覆盖服务能力、模型服务、测试监控、资源管理、知识库建设五大核心方向智能化测试领域能力要求。

本标准适用于各行业企业对其组织或项目的测试能力成熟度进行评估，包括软件测试工具平台的提供方，以及软件测试体系建设方。针对不同使用场景，本文件对其技术指标的要求略有不同。

2. 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件，仅所注日期的版本适用于本文件。凡是不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

GB/T 38634.1-2020 系统与软件工程 软件测试 第1部分：概念和定义

3. 术语和定义

下列术语及定义适用于本文件

3.1

错误 error

系统运行中出现的可能导致系统崩溃或者暂停运行的非预期问题。

[来源：GB/T 39412-2020，3.1.6]

3.2

测试用例 test cases

前置条件、输入（包括操作步骤）和预期结果的集合，用于驱动测试项的执行以满足测试目标，测试目标包括正确实现、错误识别、检查质量和其他有价值的信息。

[来源：GB/T 38634.1-2020，3.48]

3.3

测试数据 test data

为满足执行一个或多个测试用例的输入需求而创建或选择的数据，该数据可在测试计划，测试用例和测试规程中定义。

[来源：GB/T 38634.1-2020，3.55]

4. 缩略语

下列缩略语适用于本文件：

API：应用编程接口（Application Programming Interface）

CI/CD：持续集成/持续交付（Continuous Integration/Continuous Delivery）

DSL：动态脚本语言（Dynamic Script Language）

5. 智能化软件测试能力成熟度模型总体架构

本文件所定义的智能化软件测试能力成熟度模型，如图1所示：



图 1 智能化软件测试能力成熟度模型

6. 能力分级

6.1. 分级概述

智能化软件测试能力成熟度模型，共分为5个级别，每个级别中按照不同程度说明。各级别对应的具体能力要求见第7-11章。各级别技术要求描述如下：

级别	中文	定义
----	----	----

1级	初探引入级	在测试过程中引入AI能力，不限于开源、自研、商用等。测试过程具有混沌性和不明确性，AI的整合和应用尚处于初步探索阶段。
2级	技术辅助级	AI 作为工具，对能力实现起辅助作用的环节由 AI 执行，测试主导权仍然掌握在测试人员手中。AI 仅起到辅助和加速测试流程的作用。
3级	场景增强级	在测试过程中，对能力实现起关键作用的环节由 AI 主导执行，与测试人员形成协同工作的模式，共同推进测试进程。
4级	自主引领级	特定条件下（业务简单/规模小/新项目等），AI 能自主完成功能、性能、安全等全过程测试。测试人员适时分配任务，必要时少量干预，如结果检查。此级别 AI 主导，测试人员监督确认。
5级	全面智驭级	AI能自主完成所有测试任务，不受业务复杂度或测试类型限制。测试人员在此级别辅助监督验证结果并偶尔干预AI无法解决的问题。AI主导测试，实现高度自主化流程。

6.2. 分级规则

智能化软件测试能力成熟度模型分级，能力要求项分为1级、2级、3级、4级、5级。均为可选项，可根据企业自身能力进行选择认证。

满足90%的1级能力项，即可认为达到初探引入级要求；

满足90%的1-2级能力项，即可认为达到技术辅助级要求；

满足90%的1-3级能力项，即可认为达到场景增强级要求。

满足90%的1-4级能力项，即可认为达到自主引领级要求。

满足90%的1-5级能力项，即可认为达到全面智驭级要求。

可选项供参考，不作为必须满足项。

7. 服务能力

7.1. 需求分析

7.1.1. 需求理解分析

1 级：

——应能够从多源数据（如需求文档/设计规格说明/会议记录/用户访谈记录等）中提取关键需求信息和功能点。

2 级：

——应能够从提取的需求信息和功能点，按照既定模板自主整理，形成需求清单（如需求的背景/目标/应用场景/测试类型等）。

3 级：

- 1) 应能够分析语义，理解并识别需求文档中的关键业务规则/功能要求/操作步骤等，确定需要覆盖的测试场景。
- 2) 应能够自动生成需求测试内容和验收条件，并进行需求强化和关键信息整合，例如功能点/性能指标/UI 要求等。

4 级：

- 1) 应能够自动分析需求中的测试点，能够自主设计测试场景，覆盖需求中的关键业务规则、功能要求、操作步骤等，以确保测试场景的全面性和有效性。
- 2) 应能够基于现有技术、资源、时间等因素，对需求进行可行性分析，识别出可能存在的技术障碍、资源限制或时间约束，为测试计划的制定提供依据。
- 3) 宜能够自动分析需求中的测试点，依据重要性、风险级别和业务影响自动排序。
- 4) 宜自主分析需求之间的关系，包括先后顺序、依赖关系等，能够生成直观的需求依赖图，明确实施顺序与业务逻辑。

5 级：

- 1) 应建立系统化的需求分析模型，具备自主收集测试决策前后的数据，实现自我学习与能力演化，持续优化需求分析过程。
- 2) 应具备完全自主地理解需求文档能力，例如识别复杂的业务逻辑/用户故事等，考虑客户群体/行业属性/软件特性/测试资源/应用场景/风险等多方面因素的影响。
- 3) 应与测试管理工具无缝对接，形成从需求分析到测试设计的自动化推进。

7.1.2. 需求质量优化

1 级：

——应能够检查需求文档，识别并标注出明显语法错误/拼写错误/格式不一致等问题。

2 级：

- 1) 应有需求质量检查标准，规范语法/拼写/格式/逻辑等方面要求。
- 2) 应自动根据规范检查需求文档，确保需求的完整性和合理性。

3 级：

- 1) 应自动验证需求文档是否全面覆盖功能和非功能需求，确保无遗漏。
- 2) 应能够检查需求是否明确业务需求/用户价值/技术实现难度等要素，确定需求优先级。

4 级：

- 1) 应能够识别需求中的隐含信息/上下文关系/可能的歧义等，优化需求质量。
- 2) 应能够进行需求一致性校验，识别单个/多个关联需求间冲突描述，优化业务一致性。
- 3) 应具备智能验证/智能模拟/测试等能力，验证需求是否符合业务目标/用户需求。

5 级：

- 1) 应对需求文档进行持续学习和优化，不断提升需求质量的分析和预测能力。
- 2) 应建立知识库，存储常见需求问题、解决方案和最佳实践，便于团队成员共享和学习，提升整体需求质量水平。

- 3) 应与需求管理流程紧密结合，实现需求从制定、检查、评估到改进的闭环管理，确保需求质量的持续优化和提升。

7.1.3. 需求变更管理

1 级：

——应引入 AI 能力辅助需求变更管理，可进行识别、记录、跟踪。

2 级：

——应能对需求变更处理流程进行智能跟踪。如按优先级、版本、产品测试维度智能识别、记录、跟踪。

3 级：

- 1) 应能够分析需求变更，识别出变更对功能、性能、安全的影响。
- 2) 应能对需求变更风险进行智能评估。

4 级：

- 1) 应能对需求变更风险智能的提出合理化建议。
- 2) 应能基于需求变更对测试计划/设计/用例/数据等提出调整建议。

5 级：

- 1) 应能实现完全智能化需求变更管理，无需人工干预。如识别、记录、跟踪和分析变更，预测变更风险，制定应对策略，并持续优化变更管理流程。
- 2) 应能与其它能力模块（如需求分析、测试计划、测试设计、测试执行等）联动，对于需求变更影响的测试资源，测试进度计划、测试用例、测试流程等能进行智能调整。

7.2. 测试计划

7.2.1. 测试策略制定

1 级：

——应具备基于规则/模板，使用数据分析工具来进行历史数据采集和分析，为测试策略（包含测试范围、测试环境、测试方法、测试时间、测试工具和技术、优先级、风险分析等）的制定提供参考。

2 级：

——应能够根据项目环境、业务背景、产品元素、质量标准，自动制定测试策略（测试方法、测试工具、测试技术等）。

3 级：

- 1) 应理解项目的具体背景和业务目标，制定精准的测试策略。
- 2) 应具备风险评估手段，识别关键风险领域，生成测试策略建议。

4 级：

- 1) 应具备针对为不同的测试类型（如功能测试、非功能测试等）精准推荐高匹配测试策略的能力。
- 2) 应支持基于历史数据和当前项目情况，自主制定测试策略。
- 3) 应具备自适应能力，能够根据软件的更新迭代自动调整测试策略，并提供决策支持。
- 4) 应支持基于多模态输入生成测试策略的能力，如需求文档，流程图等。

5 级：

- 1) 应支持自主地制定全面的测试策略，涵盖所有必要的测试活动。
- 2) 应能够持续监控项目进展并预测风险项，自主进行测试策略的调整。
- 3) 应能够根据测试结果和反馈，自动调整测试策略，提高测试效果和效率。
- 4) 应支持自动将测试策略实体化，结合测试资源规划情况智能拆解出测试任务。

7.2.2. 测试资源规划与分配

1 级：

——应引入人工智能能力，如使用数据分析工具来统计历史项目中的资源消耗，辅助规划测试所需的软件/硬件资源及人力资源分配。

2 级：

——应具备优化测试资源分配能力，例如通过调度算法来平衡测试资源等。

3 级：

- 1) 应具备根据项目测试需求，测试任务特性、优先级、团队成员技能和资源可用状态，智能分配测试任务，优化资源利用。
- 2) 应支持通过逻辑推理分析项目需求，预测资源瓶颈，并提出优化建议。

4 级：

- 1) 应支持动态调整测试资源分配，能够自动根据项目需求或资源变动重新评估并优化资源配置。同时提供调整方案及其对测试进度和质量的影响分析。
- 2) 应实时监控资源使用情况，以应对测试过程中的变化。

5 级：

- 1) 应支持完全自主地进行测试资源规划，如软件/硬件资源等全面管理，自主决策测试资源的分配策略，自主调度工具执行资源规划任务。
- 2) 应支持自主收集和分析测试资源规划前后的数据，不断优化资源规划算法和策略，实现自主能力演化。

7.3. 测试设计

7.3.1. 测试用例生成

1 级：

——应引入 AI 能力，具备通过多轮对话/辅助生成测试用例的能力。

2 级：

- 1) 应能自动匹配前序需求分析输出，生成的测试用例能有效覆盖需求规格说明中的关键功能性需求，可人工辅助设计非功能性需求。
- 2) 应具备自动生成多种测试场景，包括正向、反向测试场景、边界条件、异常情况测试用例，且用例应包括输入数据/执行条件/预期结果等内容。

3 级：

- 1) AI 生成的测试用例覆盖功能需求和非功能性需求，人工可以添加或调整测试用例以满足业务逻辑。
- 2) 应对生成的测试用例进行自动验证，确保测试用例中的步骤和预期结果符合逻辑。
- 3) 应具备测试编排技术，能够自动建立测试用例之间的关联关系，形成测试用例集，便于测试的执行和管理。
- 4) 应在测试用例智能生成时支持多种主流用例设计方法，如等价类划分/边界值分析/判定表/因果图等。
- 5) 应具备对现有测试用例进行更新扩充、改写、润色等优化能力。

4 级：

- 1) 应具备生成的测试用例有效覆盖需求规格说明中的所有功能和他非功能性需求，基本无需人工添加或者调整。
- 2) 应能够按照用例库指定模版自动生成用例，包含如测试目录划分/用例名称/预置条件/用例优先级/用例步骤等。
- 3) 应具备根据一句话描述/一组 UI 界面/前端源码，智能生成测试用例的能力。
- 4) 应能够自动分析需求中的复杂业务流程，生成相应的复杂场景测试用例，提高测试的深度和广度。
- 5) 应具备通过对需求文档、设计文档、页面源码、操作日志以及现有测试用例的综合分析，对测试用例进行智能查缺补漏的能力。

5 级：

- 1) 应提供在被测系统持续迭代过程中测试用例自动升级更新、自主演化的能力。
- 2) 能够根据不同的测试目标（如功能测试、性能测试、不同用户场景）动态生成多套测试用例方案，并推荐最优方案执行。
- 3) 应与持续集成/持续部署（CI/CD）流程无缝集成，实现测试用例的自动化生成、执行、保存和反馈。
- 4) 宜具备测试用例解释/转换/定位/修复等能力。
- 5) 宜构建一套一键生成用例的系统/平台，同时收集研发人员及测试人员对系统的使用反馈，评估用例生成的价值。

7.3.2. 测试数据生成

1 级：

——应具备基础的数据生成能力，根据人为设定的规则或基本模板生成测试数据，确保数据基本可用。

2 级：

- 1) 应具备能够基于预定义的规则或使用基本的机器学习模型等人工智能技术生成测试数据，能够具备对简单数据场景的识别能力。
- 2) 应运用反向思维技术，对测试用例和需求进行反向分析，生成多种类型的测试数据，包括正常数据、边界数据和异常数据，以覆盖更多测试场景。

3 级：

——应具备测试场景分析技术，自动分析需求业务场景，生成带有业务流程的输入数据和结果验证数据。

4 级：

- 1) 应具备智能数据处理技术，自动对生成的测试数据进行预处理，包括数据清洗、脱敏和转换，以满足测试需求。
- 2) 应利用智能分析技术，自动分析测试数据特征，优化数据生成策略，提高测试数据的针对性和有效性。
- 3) 应结合智能分析和匹配技术，自动分析历史测试数据，匹配数据模式，并据此生成新的测试数据。
- 4) 应具备根据真实业务场景动态生成生成测试数据能力。

5 级：

- 1) 应结合深度学习和自然语言处理技术，生成具有高度复杂性和真实性的测试数据，包括非结构化数据。
- 2) 应具备自动化执行过程中生成测试数据能力/根据上下文更新生成局部测试数据能力。
- 3) 应与测试环境无缝集成，实现测试数据的实时生成和注入，提高测试效率和准确性。
- 4) 应自动建立测试数据库，记录数据模板、规则、优化策略等，便于团队成员共享、学习和参考，进一步提升团队整体的测试数据生成能力。

7.3.3. 测试脚本生成

1 级：

——应引入 AI 能力，具备根据测试用例生成测试脚本（接口/UI/性能/安全等）的能力。

2 级：

- 1) 应结合人工制定的脚本优化策略，使用人工智能技术自动填充/完善基本的测试脚本，如循环、条件判断、异常处理等，增强脚本的可用性。
- 2) 应自动分析测试用例，具备生成符合业务逻辑的测试脚本的能力。

3 级：

- 1) 应在 UI 测试执行过程中使用 DSL 全面替代传统编程语言。
- 2) 应在生成的测试脚本中提供测试数据参数化及检查点灵活配置的能力。

- 3) 应具备利用自然语言处理/大语言模型等 AI 技术将单个测试用例一次性转为 UI 自动化测试脚本的能力，并且在转化过程中自适应企业自动化测试平台的语法格式。

4 级：

- 1) 应具备将多个测试用例一次性转为测试脚本的能力，并在转化过程中自动剔除无用或重复的短句/词语/文字/标点。
- 2) 应具备在测试脚本转化过程中对接本地知识库，自动更正测试脚本中页面对象标识与实际不一致之处，领域知识库包括，需求文档、设计文档、用户手册、存量测试脚本、页面源码等。
- 3) 应具备在测试脚本转化过程中，自动识别测试用例中检查点，并对应转化为测试脚本中的检查点。
- 4) 应具备在测试脚本转化过程中，自动区分测试用例中的操作步骤及测试数据，并自动对应进行参数化处理。
- 5) 应具备根据一组 UI 界面/视频/前端源码，智能生成测试脚本的能力。
- 6) 宜在业务代码生成过程中，同时生成对应的测试脚本。
- 7) 宜提供在被测系统持续迭代过程中测试脚本自动升级更新的能力。

5 级：

- 1) 应具备将整个测试用例文档一次性转为多个测试脚本的能力，并在转化过程中自动合理划分测试脚本边界、按用例文档对多个脚本进行合理编排。
- 2) 应在测试脚本生成过程中具备将测试用例文件与测试数据文件进行全自动准确参数化匹配的能力。
- 3) 宜具备测试脚本解释/转换/定位/修复等能力。

7.3.4. 测试用例评审

1 级：

——应开始引入 AI 技术辅助用例评审，如智能记录评审中的问题和建议等，结合人工判断确保测试用例的基本质量和覆盖关键功能需求。

2 级：

——应人工建立基本的测试用例评审流程，明确评审的步骤和要求。使用 AI 技术辅助评估测试用例，如是否按照既定模板编写/测试用例描述是否清晰/是否与需求目标一致/以及测试用例的期望结果是否唯一等。

3 级：

- 1) 应能够自主进行测试用例评审，自动分析和评估测试用例的质量和覆盖率。
- 2) 应具备检查测试用例与需求的一致性/检查测试用例的正确性和完整性/自动分析测试用例的覆盖范围，并提供评审结果。

4 级：

- 1) 系统应能够生成全面的评审报告，列出每个测试用例的评审结果，包括其优缺点/覆盖情况/与需求的匹配程度等。报告应提供清晰的可视化数据，帮助团队快速理解评审结果。
- 2) 基于评审结果，系统应提供具体的改进建议，指导测试人员如何优化测试用例设计。如增加缺失的测试用例/调整现有用例的步骤或预期结果/去除冗余的测试用例等。
- 3) 宜具备自动优化测试用例设计的能力，能够根据评审结果和改进建议，自动调整测试用例的结构和内容，确保测试用例的高质量。

5 级：

- 1) 应实现完全智能化的测试用例评审过程，无需人工干预。应能够自动执行评审任务，分析测试用例的质量和覆盖率，确保评审的准确性和效率。
- 2) 应能够根据不同的测试目标 and 需求，动态生成多套评审用例方案。分析各种方案的优缺点，并推荐最优方案执行，以确保测试用例的适用性和有效性。
- 3) 应具备自适应能力，能够根据测试环境和需求的变化，持续改进评审过程和测试用例设计。系统应能够学习历史评审数据，优化评审策略。

7.4. 测试执行

7.4.1. 用例/脚本执行

1 级：

——应开始引入 AI 技术辅助手工测试执行提效。

2 级：

- 1) 应具备使用 AI 技术辅助测试的能力，如辅助操作/数据填充/异常判断等。
- 2) 应具备自动构建编译脚本的能力。

3 级：

- 1) 应具备可覆盖主要页面的智能化测试执行能力，并自动收集测试结果。
- 2) 应在脚本智能化执行中具备解析自然语言脚本并驱动其跨平台稳定执行的能力。
- 3) 应具备智能识别代码中的分支、循环、函数调用等结构的能力，并根据分析结果确定可能的测试路径。

4 级：

- 1) 应具备根据优先级选择和应用最合适模型进行用例/脚本选择的能力，以提高测试的针对性和效率。
- 2) 应能根据执行错误的用例自动生成和提交缺陷。
- 3) 应能全自动的构建支持不同平台、开发工具的编译脚本，实现编译脚本的复用。

5 级：

- 1) 应具备复杂业务模式下覆盖全面业务流程的无脚本自动化测试执行能力，并且在过程中自动填充数据、自动设置检查点、自动生成报告，无需人工干预。

- 2) 应在脚本智能化执行过程中对自动化脚本提供容错能力，如更正测试语法不一致/页面对象标识错误等。

7.5. 报告与缺陷

7.5.1. 报告生成与分析

1 级：

——应能通过人员提供的要求，自动生成测试报告，不限于功能测试报告、性能测试报告、安全测试报告等。并根据不同报告生成测试结论。

2 级：

——应能根据测试类型自动对测试报告进行分析后输出分析意见用于辅助测试人员理解测试结果，发现潜在问题。

3 级：

- 1) 应在测试报告内容以图、表等可视化展示的基础上，引入智能展示方式，如虚拟机器人/智能体等。
- 2) 应能在测试报告中对已识别的故障和缺陷及其状态进行标记和记录。
- 3) 应具备将 AI 能力应用到关键测试结果分析活动中，如复杂缺陷的根因分析/测试覆盖率的评估等。
- 4) 应能根据测试需求及问题清单，自动识别潜在问题并生成改进建议。
- 5) 应结合历史测试报告，自动识别并分析测试问题趋势（如缺陷分布、性能瓶颈演变），并基于数据自动生成决策意见。

4 级：

- 1) 测试报告应评估已识别问题，风险等级，并生成详细风险评估报告，涵盖如风险预测、可能的影响范围、缓解措施等。
- 2) 应能提供智能体在测试过程中完成测试执行、异常检查、日志收集、报告生成等工作。
- 3) 应能利用智能问数技术对组织内多项目、多团队的历史测试结果进行自助分析。

5 级：

- 1) 应能对测试结果进行多维度（如性能指标、用户行为模式、系统负载等）可追溯性分析，如测试结果分别与测试需求、测试用例、测试代码等之间的追溯。
- 2) 应具备自我学习能力，能够基于新的测试数据和反馈，不断优化结果分析模型，提高分析的准确性和效率。
- 3) 应具备实时分析测试数据能力，通过人工提问方式提供实时的测试优化建议。
- 4) 宜建立桌面虚拟助手，实现测试过程交互以及测试结果的实时反馈。

7.5.2. 缺陷预测

1 级：

——应基于历史测试数据（如缺陷报告、测试日志等），初步使用人工智能技术，如简单的统计方法或基于经验的规则，辅助生成初步的缺陷预测结果，仅作为测试人员参考。

2 级：

——应利用人工智能技术自动收集缺陷数据，如缺陷报告、代码审查、测试日志等数据源中提取缺陷信息，并将缺陷按照类型（如功能缺陷、性能缺陷、安全缺陷等）进行自动分类，为后续分析提供基础数据。

3 级：

- 1) 应基于全面的历史测试数据，包括代码质量指标、测试执行结果等，利用机器学习算法或自然语言处理（NLP）技术，构建预测模型，给出更准确的缺陷预测结果。
- 2) 应具备智能分析技术，自动分析采集版本控制历史、系统日志、业务代码，自动生成分析缺陷引入的原因，生成根源分析报告。

4 级：

- 1) 应构建自适应的预测模型，使用深度学习算法（如神经网络）等技术，能够实时学习新数据并更新预测结果，可用于实时监控测试进度和质量。
- 2) 应自动分析代码后生成缺陷预防建议，如代码重构、测试策略优化等，帮助团队提升代码质量和测试效率。
- 3) 宜具备性能表现预测能力，通过历史测试数据及监控数据（如不同并发用户数下的响应时间、吞吐量、错误率）建模，预测新需求场景下的性能表现及性能瓶颈缺陷。

5 级：

- 1) 使用高度智能化的预测模型，能够自动学习、优化和适应新环境，考虑多种因素（如项目特性、团队经验、测试策略等），输出高度准确的、实时的缺陷预测结果。
- 2) 宜能与开发环境和版本控制系统无缝集成，实时监控代码质量，提前发现潜在缺陷，降低缺陷修复成本，确保软件质量的持续改进。

7.5.3. 缺陷跟踪与管理

1 级：

——应引入 AI 技术辅助缺陷状态的跟踪与管理，实现缺陷生命周期的可视化。

2 级：

- 1) 应具备自动采集缺陷信息的功能，并识别常见的缺陷模式和类型。
- 2) 应能够根据缺陷描述和历史数据，自动分类并排序缺陷的优先级和严重性等级。

3 级：

- 1) 应支持缺陷的智能追踪能力，自动更新缺陷状态和相关信息，确保缺陷得到及时处理。
- 2) 应能根据缺陷优先级及严重性等级智能预估缺陷修复所需的测试资源和修复时间。
- 3) 应能根据缺陷严重程度智能分配任务并提交给相关人员处理。

4 级:

- 1) 应通过学习历史缺陷数据和解决方案, 分析当前缺陷的根因, 并提供解决方案建议。
- 2) 应能自动对未修复的缺陷重新进行优先级排序与严重性评估, 并智能分配任务给相关人员。
- 3) 应能对缺陷生命周期各环节的测试资源、时间等进行智能分析, 提出缺陷跟踪和管理流程优化建议。

5 级:

- 1) 应实现从缺陷识别、记录、分类、优先级排序、修复和验证的全链条智能化跟踪与管理。
- 2) 应具备缺陷数据实时更新, 能够根据缺陷分析结果智能地新增或调整现有缺陷状态, 同时自动完善缺陷信息。
- 3) 应能根据历史缺陷修复相关数据, 并与其它能力模块(测试计划、测试设计、测试执行、报告与缺陷、回归测试等)联动, 构建缺陷修复智能处理模型。

7.6. 回归测试

7.6.1. 回归测试执行

1 级:

——应利用人工智能, 如机器学习等, 构建回归测试用例集, 辅助建立基本的回归测试计划。

2 级:

——应能够根据程序的变动情况对测试用例的优先级进行分析和打分, 筛选出优先需要执行的重要用例。

3 级:

- 1) 应能够进行基于需求文档和代码变更的影响面分析, 明确回归测试范围和预期目标。
- 2) 应能够动态更新回归测试用例。

4 级:

- 1) 应具备实现端到端测试能力, 从数据准备、测试执行到结果分析的智能化全流程。
- 2) 应能够自动检测代码变更情况, 并给出自动化调整测试用例/数据, 如入参变更、数据结构变更等。
- 3) 应能根据现有测试资源及测试周期约束, 智能推荐最小回归测试用例集。
- 4) 应能在回归测试完成后, 自动输出测试有效性度量数据。

5 级:

- 1) 应具备 AI 驱动决策能力, 通过人工智能技术让 AI agent 自行决定何时、何地进行何种类型的回归测试。
- 2) 宜具备完整的回归测试框架平台/构建回归测试系统。

7.6.2. 修复验证

1 级:

——应引入智能能力辅助修复验证，如在开发人员提交修复后，使用 AI 工具自动更新缺陷报告状态，并记录修复进度。

2 级：

- 1) 应能够根据缺陷描述和关联代码智能确定验证范围和测试用例推荐，辅助测试人员进行测试用例选择。
- 2) 应能够通过分析缺陷日志、历史数据和类似缺陷修复记录，帮助快速定位并提出修复建议或给出示例修复代码。
- 3) 应能基于回归测试结果评估回归测试用例是否已全部覆盖变更代码范围。

3 级：

- 1) 应能对回归测试反馈的异常/问题进行智能根因分析，对造成问题的原因进行分类。
- 2) 应能对回归测试的性能指标变化进行智能根因分析，评估修复是否对性能产生负面影响。

4 级：

- 1) 应能够在开发人员提交修复代码后，自动选取测试用例，触发验证测试，验证修复效果。
- 2) 应能对回归测试反馈的异常/问题进行智能分析并给出缺陷修复建议，如通过知识图谱给出常见问题的解决方案。
- 3) 应能够根据验证测试结论自动更新缺陷状态，并跟踪修复进展。
- 4) 应能对回归测试未覆盖代码进行智能分析，并给出测试用例、测试数据的增补建议，如通过遗传算法，根据测试反馈的结果对测试用例进行调整。

5 级：

- 1) 应能实现完全智能化的修复验证，无需人工干预。
- 2) 应能基于修复验证的相关记录进行智能学习，优化回归测试执行能力。
- 3) 应能对回归测试反馈的异常/问题进行智能分析并给出缺陷修复建议。

7.7. 测试准入准出评审

下文“准入”指测试启动前需满足的条件，“准出”指测试完成后可交付的标准。

1 级：

——应引入 AI 技术辅助提测准入阶段和测试准出阶段的检查/评审工作，如智能检查准入材料完备性，结合人工判断测试（自测阶段、系统测试阶段、验收测试阶段等）证迹质量及覆盖度等是否达标。

2 级：

——应人工建立基本的测试准入准出标准流程，明确检查的材料类型和内容要求。使用 AI 技术辅助综合评估需求及设计文档质量/代码质量/开发自测质量（主流程及核心功能点覆盖情况）/测试报告质量/缺陷解决情况等，是否达到预设的准入准出标准，并给出智能分析建议。

3 级：

- 1) 应能够自主按照系统测试准入准出规范要求，进行相关材料完备性（数量达标）和质量度（质量达标）的检查。
- 2) 应具备需求文档及设计文档质量/代码质量/开发自测质量/测试报告质量/缺陷解决情况等的综合评估及检查，并提供检查结果作为后续测试及发布活动依据。

4 级：

- 1) 能够自主生成全面的准入准出检查结果报告，明确列出缺少或不合格的资料项，如：准入阶段未提供研发冒烟测试通过报告、冒烟测试报告缺少主流程验证案例、单元测试通过率不达标、需求文档缺少业务背景需求描述等。准出阶段高优缺陷未全部关闭、测试报告缺少风险告知项等。报告应提供清晰的可视化数据，帮助团队快速理解检查结果。
- 2) 基于评审结果，系统应提供具体的改进建议，指导开发、测试人员如何补充、优化准入准出项。
- 3) 宜具备环境与依赖项的预检能力，确保测试环境及依赖项在测试准入阶段即与提测要求一致，避免测试执行阶段再进行环境的搭建部署和环境问题的解决。

5 级：

- 1) 应实现完全智能化的准入准出检查全过程，无需人工干预。应能够集成至测试平台自动执行提测准入项和测试完成准出项的检查，确保准入准出检查的准确性和效率。
- 2) 应能够根据不同的需求类型和测试目标，动态裁剪准入准出标准，如：对于简易敏捷需求，需求文档内容可简化、设计文档可裁剪、测试报告可简化等，以确保迭代效率。
- 3) 应具备自适应能力，能够根据提测资料和测试证迹动态更新检查结论，并能够学习历史标注数据，优化检查策略。

8. 模型服务

8.1. 模型适配

1 级：

——应具备模型的基础集成，引入适用于软件测试的基础人工智能模型，模型的构建主要依赖于手动配置和调整，工具与测试场景的匹配主要依赖预设规则，适应性有限。

2 级：

- 1) 应对模型的基础和条件具有一定的规划，应识别出目标场景、目标任务或要求，制定适配策略和计划；
- 2) 应利用工具或技术对模型结构进行对比，提供出不同模型的差异性信息，但对业务逻辑层面的变化理解有限，需要人工干预，对出现的误判和无法完全理解的复杂模型变化进行人工审核和调整；

- 3) 应对模型参数、模型结构进行调整，训练特定的任务数据，完成主要流程的模型选择，可人工干预以处理复杂或不常见场景。

3 级：

- 1) 应具备模型集成与协同，将多个模型进行集成和协同工作，以形成更智能化的测试系统。
- 2) 应建立自适应机制，模型能够根据测试过程中的实际情况进行动态调整和优化。
- 3) 应对模型适配具有一致的理解 and 操作方法，适配的模型在性能、效率及适用性上具有一定提升，但模型适配的准确性仍需要人工驱动和监测。
- 4) 模型应具备智能决策，如根据测试结果自动调整测试策略、优先处理高风险问题等，但核心决策过程仍需人工监督。

4 级：

- 1) 应能自动解析模型的整体结构，深入理解模型在业务、架构、数据等多个维度的变化，可自动调整模型适配过程中出现的问题。
- 2) 应实现模型与系统平台的无缝集成，使模型能够自动地参与到测试过程中，无需人工干预。同时，建立自动化机制，使模型能够自动地收集和处理测试数据，以支持持续改进和优化。
- 3) 应具备反馈与迭代，建立智能反馈机制，使模型能够根据测试人员的反馈进行快速迭代和优化。同时，自动调整模型的参数和结构，以进一步提高其性能和准确性。

5 级：

- 1) 应具有智能化的系统适配方法，可实现实时无缝自主动态模型适配，为模型适配提供全面的解决方案。
- 2) 模型适配能力不仅能应对当前应用需求，还可以预测未来的发展和挑战，可根据丰富的适配架构体系做出最优的模型适配决策；
- 3) 应能够自主地进行演化和优化，不断优化测试流程和模型性能，通过持续优化模型适配度，使模型适配的精准度和稳定性不断提高。

8.2. 模型调度

1 级：

——应基于人工提供的调度要求或规则，自动执行单一模型调度流程。

2 级：

- 1) 应具备根据不同的测试任务需求制定基本的调度策略，支持在多个测试场景中自动进行模型调度。
- 2) 应具备在调度过程中能够自动收集并反馈模型执行结果，但对于调度异常和资源冲突问题，提供优化建议但仍需依赖人工干预进行处理。

3 级：

- 1) 应具备根据任务需求和系统负载自动优化调度策略，提升资源利用率和执行效率。
- 2) 应具备动态调度能力，能够实时感知任务和资源需求的变化，并自动调整调度策略以应对任务优先级和资源状况的变化。

4 级：

- 1) 应具备智能调度决策机制，自动根据任务特征、模型执行状况和系统负载自动优化调度策略，自动进行资源预测和优化调度，自动处理复杂的调度问题。
- 2) 应具备多模型协同调度能力，能够自动分配任务给不同模型，自动协调多任务之间的资源和调度冲突，逐步向全自动调度系统过渡。

5 级：

- 1) 应具备持续学习能力，新的模型和调度环境都可以适应被测系统，实现模型的自动优化和策略的自我完善。
- 2) 应拥有完全的自主决策能力，能够在无需人工干预的情况下，全自动执行模型的选择和调度流程。

8.3. 模型微调

1 级：

——应基于开源模型平台通过人工提供的要求、测试数据等自动执行模型训练或微调。

2 级：

- 1) 应能根据任务需求，智能评估并选择适合的预训练模型。
- 2) 应能根据任务需求，智能选择用于模型微调的数据集。
- 3) 应能根据设定的微调目标对模型输出进行智能评估，判断是否达到目标，若达到目标则提前终止训练。

3 级：

- 1) 应具备机器学习算法/智能分析等能力。自动检测微调过程中的异常情况。自动进行模型训练或者微调。
- 2) 应结合测试时人工反馈数据和实时业务需求，自动调整微调方案与参数选择，使模型在多变测试任务中持续优化。
- 3) 应能根据任务需求以及所选预训练模型，智能给出模型微调策略，如调整模型架构、调整超参数等。
- 4) 应能使用验证集定期自动对模型训练过程进行监控，智能评估是否过拟合，并给出调整建议。
- 5) 应能使用测试集对模型性能进行智能评估，评估其在新数据上的泛化能力，并给出调整建议。

4 级：

- 1) 应具备学习能力，通过人工反馈结合历史测试结果自动学习微调策略，自动进行微调。
- 2) 应具备自主选择、清洗与增强微调数据集能力，自动进行数据清洗，减少不相关数据对微调效果的干扰。

5 级：

- 1) 应利用大规模数据分析和预测模型，提前判断未来测试需求趋势与潜在问题，预先进行相应的微调准备。

- 2) 应能于持续集成工具结合实现端到端的自动化优化流程，包括数据预处理、模型训练、验证和部署。

8.4. 安全管控

1 级：

——应引入人工智能能力辅助模型安全管控，帮助模型在运行过程中保持安全、可靠和合规。

2 级：

- 1) 应采用人工智能技术对模型的输入进行加密、匿名化和脱敏处理等前置处理。
- 2) 应能根据模型推理目标智能建立敏感数据评估体系，并进行访问权限控制。
- 3) 应能自动在模型中嵌入数字水印、用户虚拟形象等标记，用于追踪未授权的使用。
- 4) 应能智能识别未授权访问、异常输入等，并给出提示。

3 级：

- 1) 应能智能选取适当的加密技术保护模型参数。
- 2) 应能记录模型训练、微调、推理过程的详细信息，智能建立数据间的关联关系，便于后续审计和问题排查。
- 3) 应能对出现的安全问题进行智能的追溯以及根因分析。

4 级：

- 1) 应能智能的定期对模型的安全性和合规性进行审计，并提出合理化建议。
- 2) 应能智能建立对抗样本检测机制，识别并抵御针对模型的对抗攻击。
- 3) 应能智能的定期对模型进行鲁棒性测试，评估其在面对各种攻击手段时的表现。

5 级：

- 1) 应能实现完全智能化的模型安全管控，无需人工干预。
- 2) 应能与模型监控、数据管理等能力模块联动，确保模型运行的安全、可靠、合规。
- 3) 应能结合多种模型或算法进行智能决策，提高模型的鲁棒性和安全性。

9. 测试监控

9.1. 测试执行监控

1 级：

——应能对测试执行过程中程序运行是否正常进行智能记录和展示。

2 级：

- 1) 应能通过语音/文字等对测试执行过程中的测试相关信息进行智能采集和展示，如测试时间、测试人员、测试用例等。
- 2) 应能对监控数据（设定的对测试进度进行监控的数据，如代码结构覆盖率等）进行智能图形展示。如进行指定监测对象在指定时间范围内对测试功能点的聚类展示。

3 级:

- 1) 应能智能化（如通过数据分析与挖掘技术等）建立并展示监控数据之间的关联关系。
- 2) 应能对测试执行过程中产生的代码覆盖数据进行智能采集和展示。
- 3) 宜智能生成函数调用图/控制流程图来展示代码覆盖情况。
- 4) 宜能根据监控权限智能化展示多系统不同层次代码结构覆盖信息。

4 级:

- 1) 应能对多系统共同测试执行过程进行智能化的统一监控。
- 2) 应能对多系统监控数据以统一的标准进行汇总和展示。
- 3) 应能智能定位并展示测试现场故障和缺陷逻辑在代码中的位置。
- 4) 应能对监控数据进行智能分析和展示，如聚类分析、漏洞分析等。

5 级:

- 1) 应实现多系统全流程的测试进度智能监控，无需人工干预。
- 2) 应能根据历史监控数据，对测试进度监控能力进行智能优化。
- 3) 宜实现根据不同角色人员的监控需求对监控数据分别进行分析处理，并将处理结果及建议实时反馈给对应人员。

9.2. 模型监控

1 级:

——应引入人工智能能力辅助测试模型监控，帮助提高模型性能/稳定性/准确性。

2 级:

- 1) 应能记录和评估模型输入和输出的有效性和安全性，并给出提示。
- 2) 应能监控和记录模型运行状态，记录异常并给出提示。
- 3) 宜能实时监控并记录模型运行时的资源与成本消耗，并给出提示。

3 级:

- 1) 应能评估模型输入和输出的准确性，并给出提示。
- 2) 应能监控和记录模型运行时的性能参数，并给出提示。

4 级:

- 1) 应能基于模型输入和输出的监控数据，提出合理建议。如通过对监控数据进行智能评估得出模型输入/输出的数据不准确，提出改进数据标注质量/模型微调的改进建议。
- 2) 应能基于监控数据评估模型的有效性和效率，并提出合理建议。
- 3) 应能建立起预警机制，当发现模型出现异常情况时，及时发出预警并给出决策建议，如暂停测试、重新训练模型或者调整测试策略等。

5 级:

- 1) 应能实现完全智能化的模型监控，无需人工干预。
- 2) 应能与其它能力模块（如模型适配、模型调度、模型微调等）联动，进行模型的智能优化。
- 3) 应能根据历史模型监控数据，对模型监控能力进行智能优化。

9.3. 资源监控

1 级:

——应引入人工智能能力辅助测试资源监控，帮助评估关键资源状态/使用情况/性能等。

2 级:

- 1) 应能智能化（通过语音、文字等指示等）设置测试资源的性能参数的告警范围。
- 2) 应能对测试资源的性能参数进行采集与展示，并在参数超出告警范围时智能的发送通知。
如通过语音、文字等指示对指定时间范围内指定监控对象进行监控数据采集与展示，参数超出范围时通过智能语音发出提示。

3 级:

- 1) 应能定期对资源监控数据进行汇总展示。如根据测试资源的不同类型，智能的选择合适的形式进行数据的汇总展示，便于信息的获取和挖掘。
- 2) 宜能对监控到的资源故障进行智能化分级、标注与发送。

4 级:

- 1) 应能实现实时测试监控采集与展示，必要时可进行人工干预；
- 2) 应能对监控到的资源故障进行智能定位。
- 3) 应能对资源监控数据进行智能评估分析，对故障进行预测。
- 4) 应能根据资源监控数据对测试资源的性能参数参考标准进行智能化调整。

5 级:

- 1) 应能实现完全智能化实时测试监控采集与展示，无需人工干预。
- 2) 应能对监控到的资源故障的原因及其影响进行智能化的分析，并提出合理的改进建议。
- 3) 应能与其它能力（如测试资源规划）及历史资源监控数据联动，实现测试资源的智能调度。

10. 资源管理

10.1. 工具管理

1 级:

- 1) 应具备测试工具的初步自动化部署和配置的能力，如使用脚本自动安装常用的测试工具，减少手动部署安装。
- 2) 应支持测试工具的版本控制和兼容性检查，确保工具与现有系统环境的兼容性，并实现多版本管理。

2 级:

- 1) 应具备测试工具的深度自动化部署和配置能力，实现测试工具的一键安装配置。
- 2) 应具备测试工具使用情况（如工具的使用频率、执行时间）及相关信息（如版本/型号、工具操作说明、用户手册等）统计能力。
- 3) 应能按照功能、技术特性等对工具进行智能区分并对工具信息分类保存。

- 4) 应实现测试工具的自动化配置验证和最小化部署，以提高部署效率和减少资源消耗。
- 5) 应增加测试工具性能监控能力，实时跟踪工具的运行状态和性能指标。
- 6) 应强化工具信息管理，包括工具的依赖关系管理、自动更新、补丁应用以及许可管理。

3 级：

- 1) 应具备测试工具适配测试任务的能力，根据不同的测试项目 and 需求，自动推荐合适的测试工具，并进行相应的参数配置。
- 2) 应能按照功能、技术特性等对工具进行智能比对。
- 3) 应能对工具问题处理流程进行智能跟踪。
- 4) 应能对工具的变更进行智能管控。
- 5) 应通过机器学习技术预测项目需求，自动推荐最合适的工具，并进行参数优化。
- 6) 应增加工具间的集成能力，支持不同工具间的接口对接和数据交换。
- 7) 应引入自动化的问题解决机制，通过人工智能技术分析工具问题，自动推荐解决方案或自动修复常见问题。
- 8) 应增加对工具变更的影响分析，评估变更对测试流程和结果的潜在影响，并提供变更回滚的能力。

4 级：

- 1) 应具备测试工具自我优化和升级的能力。自动调整参数、更新算法等，以提高测试效率和准确性。
- 2) 应能对工具的异常/问题进行智能分析，给出工具的使用限制并提出合理的建议。
- 3) 应能定期对工具进行智能巡检，确保工具的有效性和技术先进性。
- 4) 应包括智能化的测试工具评估，定期对工具的有效性进行评估，并根据评估结果进行工具的替换或升级。
- 5) 应增强工具异常分析能力，通过深度学习技术分析工具日志和错误信息，自动定位问题根源，并提供详细的故障报告。
- 6) 应增加智能化的测试工具预测维护，通过分析工具的使用模式和性能趋势，预测潜在的故障并提前进行维护。

5 级：

- 1) 应具备智能化测试工具管理平台，根据不同的测试策略，自动进行测试工具选择、部署、配置。
- 2) 应具备自主学习和优化的能力，对现有工具链进行合理调整和优化，确保工具管理体系与项目需求和技术发展相匹配，并持续保持高效运行。
- 3) 智能化测试工具管理平台应支持云原生部署，能够根据测试需求动态扩展或缩减资源，实现高度的弹性和可伸缩性。
- 4) 自主学习优化能力应包括对测试工具使用模式的学习，自动调整工具配置以适应不同的测试负载和性能要求。
- 5) 应增加对工具链的安全性评估，定期对工具链进行安全审计，确保工具的安全性和合规性。

10.2. 环境管理

1 级:

- 1) 应具备系统环境的初步自动化部署、配置和维护的能力。
- 2) 应具备基本的环境监控和预警系统能力，但需要人工介入处理问题。
- 3) 应实现环境配置的模板化管理，以快速复制和部署测试环境。
- 4) 应具备环境健康检查的基本自动化能力，定期自动执行环境检查并生成报告。

2 级:

- 1) 应具备全流程自动化部署、配置和管理系统环境的能力。
- 2) 应具备完全的自动化监控与预警系统的能力，能够快速定位故障点解决问题。
- 3) 应支持环境部署的滚动更新和蓝绿部署，以实现零停机部署。
- 4) 应实现故障自愈能力，自动重启或替换故障组件，减少人工干预。

3 级:

- 1) 应具备智能化部署、配置和管理系统环境的能力，引入机器学习和人工智能优化资源配置。
- 2) 应具备智能化监控、自动恢复系统环境的能力。
- 3) 应引入环境性能预测，基于历史数据和当前趋势预测资源需求，提前进行资源扩缩容。
- 4) 应实现环境配置的自动备份和恢复能力，以快速响应环境故障。

4 级:

- 1) 应具备根据不同需求规模和测试策略动态调整环境部署、配置、资源分配的能力。
- 2) 应具备智能分析测试结果和环境变化趋势的能力，为测试策略提供决策支持。
- 3) 应支持环境的多租户管理，确保不同测试项目环境的隔离性和安全性。
- 4) 应实现环境变更的自动化测试，确保环境变更不会影响测试结果的稳定性。

5 级:

- 1) 应具备完全智能化的自适应和自我学习的能力，动态管理系统环境。
- 2) 应具备与测试工具、测试数据和测试流程智能交互的能力。
- 3) 应实现环境的自适应安全策略，根据测试数据和行为模式动态调整安全设置。
- 4) 应支持环境的智能容量规划，基于测试需求和历史使用模式自动调整资源分配。

10.3. 数据管理

1 级:

- 1) 应引入智能能力辅助数据管理，如进行多源数据（实际用户数据/模拟数据/历史数据等）的自动化采集和存储，及安全保护。
- 2) 应实现数据的自动分类和元数据管理，以支持数据的快速检索和有效组织。
- 3) 应支持数据的自动压缩和去重，以优化存储效率。

2 级:

- 1) 应引入 AI 能力辅助进行数据清洗, 去除噪声数据、缺失值和异常值等。
- 2) 应具有数据的自动化集成和共享能力, 辅助数据融合和交互, 支持业务应用。
- 3) 应具备检查数据安全性和合规性能力, 如数据文件格式、数据范围等。
- 4) 应引入数据质量评分系统, 自动评估数据集的完整性和一致性。
- 5) 应支持数据的自动化加密和脱敏处理, 以增强数据安全性。

3 级:

- 1) 应建立统一的数据标注规范和流程, 并对数据进行分析, 具备自动化的数据标签推荐、数据分类、自动标注、版本管理等能力。
- 2) 应能对同类型数据按照统一的规范进行存取、查询和展示。
- 3) 应能对数据的表示形态进行智能转换, 如序列、AST、CFG 等, 以满足分析处理的需要。
- 4) 应实现数据的自动化异常检测, 及时发现并处理数据集中的异常值。
- 5) 应支持数据的多维度分析和可视化展示, 以便于理解和利用数据。

4 级:

- 1) 应具备定期数据质量管理能力, 支持对数据质量及合规性监控和改进自动化。
- 2) 应采用人工智能技术进行异常数据分析和隐私保护, 提供数据安全、隐私保护等能力。
- 3) 应实现数据的智能化推荐系统, 根据测试需求自动推荐最相关的数据集。
- 4) 应支持数据的自动化生命周期管理, 包括数据的创建、维护、归档和销毁。

5 级:

- 1) 应建立完整的智能化数据管理生态, 并根据实际测试需求, 进行动态调整数据管理策略。
- 2) 应使用自适应安全架构等, 确保数据的合理使用和保护。
- 3) 应支持跨平台数据管理能力。
- 4) 应实现数据的智能化审计跟踪, 记录数据访问和变更历史, 以支持合规性和溯源需求。
- 5) 应支持数据的自动化隐私影响评估, 确保数据处理活动符合隐私保护法规。
- 6) 应实现数据的智能化优化建议, 根据数据分析结果自动提出数据管理和测试策略的改进建议。

11. 知识库建设

1 级:

- 1) 应能够建立涵盖智能化测试过程中各类知识的知识库, 如测试方法、测试技术、测试工具等。
- 2) 应实现需求条目、测试用例、测试资产等核心内容的元数据标准化建模, 支持基础分类检索
- 3) 宜采用结构化的方式组织和存储知识, 如使用标准化的元数据模型、本体论等。

2 级:

- 1) 应能够建立起知识库的智能搜索、查询和推荐机制, 实现知识的自主学习和高质量推荐。
- 2) 应能对知识库入库内容和格式等进行智能管控。

3) 应能对知识库内容进行智能分类并标注。

3 级:

- 1) 应能够制定知识库的维护和更新机制, 确保知识库的时效性和准确性。
- 2) 应能智能建立知识库内容间的关联关系, 对查询结果进行关联推荐。
- 3) 应能支持对知识库内容的智能查询、展示和导出。

4 级:

- 1) 应能够将知识库与智能化测试模型相结合, 实现知识驱动的智能化测试。
- 2) 应能智能评估知识库内内容的合理性及合规性。
- 3) 应定期对知识库进行评审, 并给出改进建议。

5 级:

- 1) 应能实现完全智能化的知识库建设, 无需人工干预。
 - 2) 应能与其它能力模块联动, 智能推荐知识库内容, 促进其它测试能力的优化。
 - 3) 应能根据历史数据, 实现对知识库建设能力的智能优化。
 - 4) 应建立知识价值评估模型, 量化测试资产复用对测试效率、缺陷逃逸率等指标的提升效果。
-